Preparación paso a paso (Windows)

Solo Windows 10 (1809 o superior) y Windows 11. La guía interactiva con comandos para copiar está en `03-Entorno/guia-instalacion/index.html` (ábrela en el navegador).

Estado: borrador. Los comandos de winget **no se han probado** en un equipo Windows limpio. La lógica de los dos scripts se probó con un winget falso, no en Windows real.

Dos caminos

1. **Todo en uno:** un script instala lo que falte con winget y genera un informe. Hay dos versiones equivalentes: `instalar_entorno.cmd` (CMD) e `instalar_entorno.ps1` (PowerShell), en `03-Entorno/scripts/`.
2. **Paso a paso:** un comando por programa (winget) con enlace de descarga alternativo.

Si no puedes instalar programas (equipo corporativo), usa la **ruta sin instalación**: Codespaces y Cloud Shell desde el navegador.

Qué se instala en tu equipo

Git, Visual Studio Code (con 7 extensiones), Miniconda, DB Browser for SQLite y Google Cloud CLI. **Docker y Kubernetes no se instalan**: se usan dentro de Codespaces. Docker Desktop local es opcional.

Pasos

1. Abre CMD (Windows + R, «cmd») o PowerShell. No hace falta administrador.
2. Guarda el script de tu terminal (o usa el del repositorio).
3. Ejecuta la instalación y después la verificación.

CMD:

```
instalar_entorno.cmd
instalar_entorno.cmd verificar
```

PowerShell:

```
powershell -ExecutionPolicy Bypass -File .\instalar_entorno.ps1
powershell -ExecutionPolicy Bypass -File .\instalar_entorno.ps1 -Verificar
```

`-ExecutionPolicy Bypass` vale solo para esa ejecución y no cambia la configuración del equipo. 4. Cierra y abre la terminal antes de verificar, para que el sistema reconozca lo recién instalado. 5. Clona el repositorio y ábrelo:

```
git clone --branch develop https://github.com/helicr/curso-ml-ia-desarrollo-produccion.git
cd curso-ml-ia-desarrollo-produccion
code .
```

Nota: hasta fusionar `develop` en `main`, el contenido está en la rama `develop`; por eso el comando lleva `--branch develop` (un `git clone` a secas te deja en `main`, casi vacía). Si ya clonaste, usa `git fetch origin` y `git switch develop`. 6. **Términos de servicio de conda.** El Miniconda de Anaconda se niega a crear entornos hasta aceptar los términos de sus canales por defecto (`CondaToSNonInteractiveError`), aunque el entorno use solo conda-forge. Configuración que lo evita (probada en Linux):

```
conda config --add channels conda-forge
conda config --remove channels defaults
conda config --set channel_priority strict
conda config --add default_channels https://conda.anaconda.org/conda-forge
```

Alternativas: aceptar los términos con `conda tos accept` (revisa si tu organización necesita licencia) o usar Miniforge. 7. Cuando el instructor lo indique, crea y comprueba el entorno (en «Anaconda Prompt», desde la carpeta del repositorio):

```
conda env create -f 03-Entorno\conda\environment.yml
conda activate env-ia-mlops
python 03-Entorno\conda\prueba_de_humo.py
```

La última línea debe ser `HUMO OK`. El entorno opcional (red neuronal y LLM) se llamará `env-ia-opcional`. 8. Dentro del Codespace del curso (cuando se publique): `docker version`, `docker run --rm hello-world`, `kind version`, `kubectl version --client`.

Google Cloud

```
gcloud auth login
gcloud config set project TU_ID_DE_PROYECTO
gcloud config get-value project
gcloud auth list
```

Sustituir TU_ID_DE_PROYECTO. Verificar términos de facturación y elegibilidad. No compartir tokens. No crear recursos de pago en S0. Alternativa: Cloud Shell desde el navegador.

Si algo falla

Envía al instructor el contenido de `verificacion-entorno.txt` (lo genera el script) y el mensaje de error completo. No incluyas contraseñas ni tokens.